Accelerating Polynomial System Solving via factorization and UNSAT Core Learning

Ilay Menahem, Michael Shalyt, Ido Kaminer

August 11, 2026

Abstract

Polynomial systems are a fundamental language for describing algebraic and geometric problems across mathematics and its applications. Beyond finding individual solutions, one often seeks global properties of the solution space, such as its dimension and irreducible components, or structural properties and decompositions of the underlying polynomial ideal. For large polynomial systems with many variables and high degrees, determining these global properties can become intractable. Leading approaches include variants of Buchberger's algorithm with syzygy, decomposition techniques, resultants, and cylindrical algebraic decomposition. However, even the most modern approaches still often fail on practical problems or slight perturbations of standard benchmarks. Here we introduce a framework that divides a polynomial system into smaller branches. It assumes only that the branch solver describes satisfiable branches exactly and certifies unsatisfiable ones. If the solver also returns an unsatisfiable core, the framework uses it to prune later branches, following the same basic idea as conflict-driven clause learning (CDCL). In our experiments, we use Gröbner-basis solvers and apply the method to two systems arising from research in quantum field theory for which no standard method in our comparison succeeds. At a later stage, we construct a decision tree guided by domain-informed hypotheses to efficiently enumerate candidate solutions. Together, these ideas offer a path toward the exact analysis of previously inaccessible polynomial systems.

1 Introduction

Systems of multivariate polynomial equations appear whenever a symbolic model contains nonlinear algebraic constraints. Applications include robotics, coding the-

ory, optimization, statistics, and physics [14]. In many of these problems, an isolated solution is not enough. One needs the dimension of the solution set, its irreducible components, or exact relations that hold on every solution. These are questions about the variety and its defining ideal, rather than about a single point [2, 7].

Elimination by resultants provided an early systematic way to attack polynomial systems, but the introduction of Gröbner bases gave a common algorithmic framework for elimination, ideal membership, dimension, and related tasks. Buchberger developed this framework in his 1965 thesis [1]. Later algorithms changed how the expensive reductions are organized. F4 replaces individual polynomial reductions with sparse linear algebra, while F5 uses signatures to avoid many reductions to zero [3, 4]. Primary-decomposition algorithms use Gröbner bases and factorization to separate components [5]. Numerical algebraic geometry takes a different route: homotopy continuation and witness sets provide numerical descriptions of isolated roots and positive-dimensional components [12, 13].

Worst-case complexity remains a limitation despite these algorithmic changes. Mayr and Meyer proved exponential space lower bounds for ideal-membership problems arising from polynomial ideals [10]. On concrete inputs, the term order, reduction strategy, and pair-selection rule can change the amount of intermediate work substantially. Recent learning-based work has concentrated on these choices. Peifer et al. trained a reinforcement-learning policy for pair selection in Buchberger's algorithm [11]. Kera et al. trained transformers to compute Gröbner bases for a restricted class of zero-dimensional radical ideals [6]. Both results concern controlled input distributions; a large, general ideal still requires an exact symbolic computation.

Finding one root and describing the full zero set are different computational goals. A single root does not determine whether the system has a curve of solutions, several components, or additional isolated points. Numerical irreducible decomposition was developed in part to handle such positive-dimensional and mixed-dimensional sets [12]. When the desired output is exact, however, the calculation still has to recover ideal-level information. A direct Gröbner-basis calculation treats the full generating set at once, even when the equations already display a useful factor structure.

Our method starts with that factor structure. Many equations in our target systems factor. If $p_i = \prod_j f_{ij}$, then every solution of $p_i = 0$ makes at least one factor f_{ij} vanish. Selecting factors that meet every equation gives a hitting set of the factor family. It is enough to enumerate the minimal hitting sets, and each one defines a smaller polynomial system. We solve these systems separately. The union of their zero sets is exactly the zero set of the original input, so the split does not discard solutions or rely on a genericity assumption.

An inconsistent branch can also provide information about later branches. If the

branch solver returns a certified unsatisfiable core, every later hitting set containing that core can be skipped. The method does not require a particular branch solver or core-extraction procedure. For our experiments, we use Gröbner-basis certificates and refine their cores with the procedure of Sun et al. [15]. Conflict-driven clause learning uses the analogous pruning rule in Boolean satisfiability [9, 8]. In our setting, the learned objects are sets of polynomial factors, and unsatisfiability is certified algebraically.

Some factor branches can still be too expensive for the chosen subsolver. After a timeout, we build a second decision tree by splitting a selected variable into the cases $x = 0$ and $x \neq 0$. We encode the second case exactly by adjoining $1 - xz = 0$ with a fresh variable z. A domain expert may have a good reason to expect that a particular variable vanishes. That expectation controls which split we try first, but it is not an assumption: both children remain in the tree, and their projected solution sets recover the parent zero set.

We prove that the factor decomposition, core pruning, and zero/nonzero splits preserve the complete solution set for any branch solver satisfying the stated conditions. In the experiments, we use several Gröbner-basis engines as branch solvers. We compare the resulting implementation with direct Gröbner-basis computation, symbolic decomposition, and numerical homotopy on generated benchmarks. We then apply it to two systems from quantum field theory for which the standard methods in our comparison do not complete. The output is an exact collection of branch descriptions for the full solution set, including positive-dimensional families.

2 Preliminaries and Notation

2.1 Notation

Let K be a field, let $\overline{K}$ be an algebraic closure of K, and let $K[x_1, \ldots, x_n]$ be the polynomial ring in n variables over K. We write the input system as

$$P = \{p_1, \ldots, p_m\} \subseteq K[x_1, \ldots, x_n].$$

We assume that each p_i is nonzero and fix a factorization

$$p_i = a_i \prod_{f \in C_i} f^{e_{i,f}},$$

where $a_i \in K \setminus \{0\}$, C_i is the set of distinct nonconstant factors appearing in the factorization, and $e_{i,f} \geq 1$ is the multiplicity of f.

For any set of polynomials $Q \subseteq K[x_1, \ldots, x_n]$, we write $\langle Q \rangle$ for the ideal generated by Q and define its zero set over the algebraic closure by

$$V(Q) = \{x \in \overline{K}^n : q(x) = 0 \text{ for every } q \in Q\}.$$

For $X \subseteq \overline{K}^n$, its vanishing ideal is

$$I(X) = \{g \in K[x_1, \ldots, x_n] : g(x) = 0 \text{ for every } x \in X\}.$$

Unless stated otherwise, satisfiability refers to solutions over $\overline{K}$. Hilbert's Nullstellensatz [2] gives

$$V(Q) = \emptyset \quad \iff \quad \langle Q \rangle = \langle 1 \rangle.$$

Equivalently, the reduced Gröbner basis of $\langle Q \rangle$ is $\{1\}$. A set $U \subseteq Q$ is an unsatisfiable core of Q if $V(U) = \emptyset$.

A hitting set of $C_1, \ldots, C_m$ is a set $H \subseteq \bigcup_{i=1}^{m} C_i$ such that $H \cap C_i \neq \emptyset$ for every i. It is minimal if none of its proper subsets is also a hitting set. We denote the collection of all minimal hitting sets of $C_1, \ldots, C_m$ by $\mathcal{H}$.

2.2 Polynomial ideals

2.3 Basic algebraic geometry

3 Method

Algorithm 1 uses a branch solver as a subroutine. Given a polynomial system P, it factors each polynomial to obtain $C_1, \ldots, C_m$ and enumerates the minimal hitting sets of these factor sets. Each hitting set is itself a smaller polynomial system. For a hitting set H, the branch solver must either describe $V(H)$ exactly or certify that $V(H) = \emptyset$. It may also return a certified unsatisfiable core $U \subseteq H$. In that case, any later hitting set containing U can be skipped. No other property of the branch solver is used in the correctness proof.

The union of the solution sets of the satisfiable branches is $V(P)$. We order the hitting sets by an estimate of the cost of solving them and skip those that contain a previously learned core.

Algorithm 1 Branching

Require: A set of polynomials $P = \{p_1, \ldots, p_m\}$ and an exact branch solver $\mathcal{B}$.

Interreduce the polynomials ▷ Optional
Initialize $C_1, \ldots, C_m$ as the sets of factors of $p_1, \ldots, p_m$
Initialize $C \leftarrow \emptyset$ an unsatisfiable cores set
Initialize $S \leftarrow \emptyset$ a solutions set
for Each minimal hitting set HS of $C_1, \ldots, C_m$ **do**
    if HS contains an unsatisfiable core from C **then**
        continue
    end if
    Use $\mathcal{B}$ to solve HS
    if $\mathcal{B}$ returns a representation of $V(HS)$ **then**
        Add the returned representation to S
    else
        if $\mathcal{B}$ returns a certified unsatisfiable core **then**
            Add the core to C
        end if
    end if
end for
return S, C

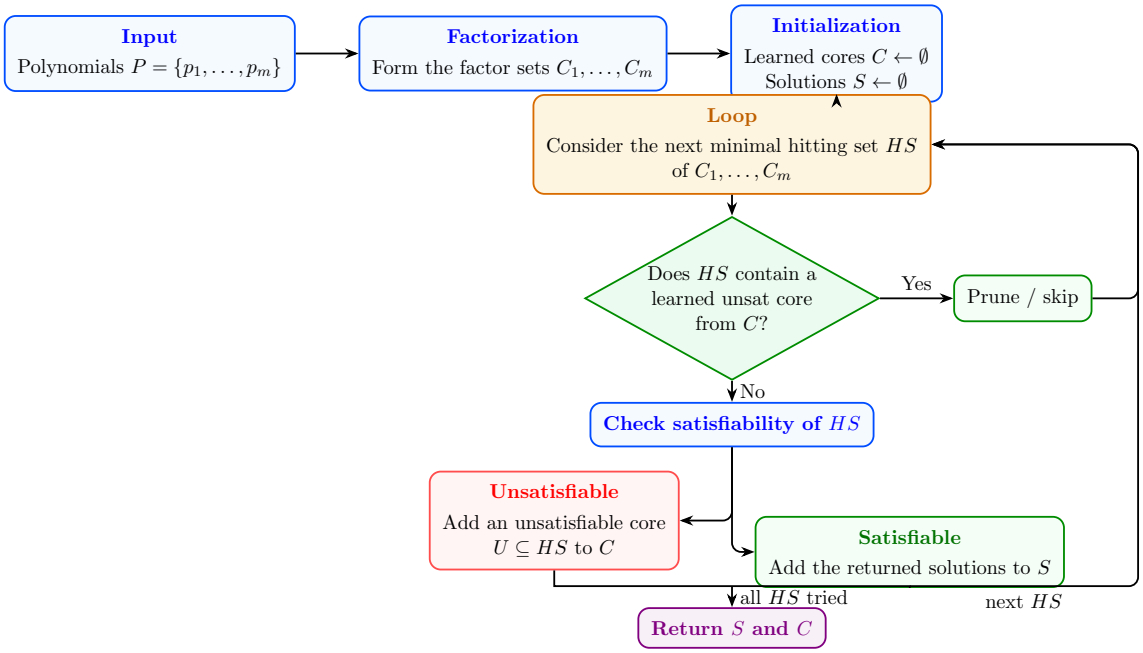


Figure 1: Flowchart of the branching procedure of Algorithm 1.

The order of the minimal hitting sets affects running time but not correctness. We use the number of factors in a hitting set as an estimate of its cost and process smaller sets first. An unsatisfiable core found in a small set may then prune larger sets before they reach the solver.

3.1 Theoretical analysis

Lemma 1 (Correctness of interreduction) *Suppose that one generator p_i is replaced by*

$$p_i' = p_i - \sum_{j \neq i} a_j p_j, \qquad a_j \in K[x_1, \ldots, x_n].$$

Then

$$\langle p_1, \ldots, p_i, \ldots, p_m \rangle = \langle p_1, \ldots, p_i', \ldots, p_m \rangle.$$

Any finite sequence of these reductions therefore preserves the generated ideal and its zero set.

Proof 1 *The polynomial p_i' belongs to the original ideal, while $p_i = p_i' + \sum_{j \neq i} a_j p_j$ belongs to the ideal generated by the updated system. Each generating set therefore*

generates the other. The same argument applies at every step of a sequential interreduction.

A zero generator imposes no constraint and can be discarded. A nonzero constant makes the ideal the unit ideal, so the input system is unsatisfiable.

Lemma 2 (Sufficiency of minimal hitting sets) *Every hitting set T of the factor family contains a minimal hitting set $H \subseteq T$, with $V(T) \subseteq V(H)$. It is therefore enough to consider minimal hitting sets.*

Proof 2 *Because the factor sets are finite, repeatedly removing redundant factors from T eventually produces a minimal hitting set H. Since $H \subseteq T$, any point in $V(T)$ also lies in $V(H)$.*

Theorem 1 *The solution set of P is*

$$V(P) = \bigcup_{H \in \mathcal{H}} V(H).$$

Proof 3 *Take $x \in V(P)$. Since $p_i(x) = 0$, at least one factor in C_i vanishes at x for every i. The set of all factors that vanish at x is therefore a hitting set of $C_1, \ldots, C_m$. Because the factor sets are finite, this set contains a minimal hitting set H. Every factor in H vanishes at x, so $x \in V(H)$.*

For the reverse inclusion, take $x \in V(H)$ for some $H \in \mathcal{H}$. The set H intersects every C_i, so each p_i has a factor that vanishes at x. Hence $p_i(x) = 0$ for every i, and $x \in V(P)$.

The same decomposition can be stated in terms of ideals.

Theorem 2 (Radical decomposition) *The radical of the input ideal satisfies*

$$\sqrt{\langle P \rangle} = \bigcap_{H \in \mathcal{H}} \sqrt{\langle H \rangle}.$$

Proof 4 *By Theorem 1, $V(P) = \bigcup_{H \in \mathcal{H}} V(H)$. Taking vanishing ideals gives*

$$I(V(P)) = I\left(\bigcup_{H \in \mathcal{H}} V(H) \right) = \bigcap_{H \in \mathcal{H}} I(V(H)).$$

Hilbert's Nullstellensatz identifies $I(V(P))$ with $\sqrt{\langle P \rangle}$ and each $I(V(H))$ with $\sqrt{\langle H \rangle}$, which gives the stated equality.

Lemma 3 *Let U be an unsatisfiable core. If $U \subseteq H$, then $V(H) = \emptyset$.*

Proof 5 *Since every point in $V(H)$ must satisfy every polynomial in U, we have $V(H) \subseteq V(U)$. The core U is unsatisfiable, so $V(U) = \emptyset$ and therefore $V(H) = \emptyset$.*

Soundness does not require U to be minimal. A smaller core is useful only because more later branches may contain it.

For the following result, assume that the branch solver terminates on every unpruned branch H. It either returns $V(H)$ exactly or certifies that $V(H) = \emptyset$. Any core used for pruning must be a certified subset $U \subseteq H$.

Theorem 3 *Algorithm 1 is sound and complete for P.*

Proof 6 *Let P' denote the system after the optional interreduction step. We check that each part of the algorithm preserves the solution set.*

Interreduction. *Lemma 1 gives $\langle P' \rangle = \langle P \rangle$, and hence $V(P') = V(P)$.*

Factorization and branch construction. *Theorem 1, applied to the factor sets of P', gives*

$$V(P') = \bigcup_{H \in \mathcal{H}} V(H).$$

Lemma 2 shows that no nonminimal hitting set can add a solution missing from this union. Thus the initial branch collection represents exactly $V(P')$.

Loop invariant. *Let R be the set of branches that have not yet been processed, and let S be the solution set accumulated so far. At the start of the loop,*

$$S \subseteq V(P') \qquad \text{and} \qquad V(P') \subseteq S \cup \bigcup_{H \in R} V(H).$$

The first inclusion is the soundness invariant. The second is the completeness invariant: every solution not yet in S remains in a pending branch. Both hold initially because $S = \emptyset$, $R = \mathcal{H}$, and the branch decomposition holds.

Core pruning. *Suppose the algorithm skips H because it contains a learned core U. Lemma 3 gives $V(H) = \emptyset$. Removing H from R therefore leaves the completeness invariant unchanged, so soundness is unchanged as well.*

Branch solving. *If H is satisfiable, the solver adds exactly $V(H)$ to S. Since $V(H) \subseteq V(P')$, the soundness invariant is preserved. Moving $V(H)$ from the pending union into S also preserves the completeness invariant. If H is unsatisfiable, then $V(H) = \emptyset$, so removing it from R preserves both invariants.*

Return. *When the loop ends, $R = \emptyset$. The completeness invariant gives $V(P') \subseteq S$, while the soundness invariant gives $S \subseteq V(P')$. Hence $S = V(P') = V(P)$.*

Note that the theoretical analysis does not require the field K to be algebraically closed. This is due to if there is no solution over the algebraic closure, then there is no solution over the field itself.

4 Branching with variable zeroing

Algorithm 2 handles timeouts from the branch solver. After a timeout, it chooses a variable x and replaces the unresolved branch with two children, one with $x = 0$ and one with $x \neq 0$. It then applies Algorithm 1 recursively to both children. The recursion stops when all branches are resolved, no splitting variables remain, or the computation budget is exhausted. This construction does not depend on the choice of branch solver.

Solutions of a Q_1 branch are projected onto the original variables, so the fresh variables do not appear in the output.

In the zero branch, the solver can substitute $x = 0$ directly. In the nonzero branch, $1 - xz = 0$ records that x is invertible. Interreduction with either equation may simplify the remaining generators. The effect depends on the branch solver. Section 5 describes the Gröbner-basis solvers used in our experiments.

The run on `systems/system.txt`, with a five-second timeout for each cover, produces the recursion tree in Figure 2. The variable chosen after a timeout is written inside the corresponding cover node. Its outgoing edges give the zero and nonzero cases.

4.1 Choice of variable for splitting

The choice of variable for splitting effects the efficiency of the algorithm. Every split creates at least two new covers to check. If a variable barely occurs in the current cover, both recursive calls may repeat most of the same work. A variable that occurs in many generators is a more promising choice: setting it to zero removes all terms containing it at once, while in the nonzero branch the algorithm can use the $1 - xz$.

Algorithm 2 Recursive branching with variable zeroing

Require: A polynomial system P, and a timeout τ.

function ZeroingBranching(P, τ)
  $C \leftarrow \emptyset$ ▷ One global unsatisfiable-core set
  $X \leftarrow$ Variables(P) ▷ All variables of P
  return SolveByZeroing(P, X, τ, C)
end function
function SolveByZeroing(P, X, τ, C)
  $(S, T) \leftarrow$ BranchingWithTimeout(P, τ, C) ▷ Algorithm 1; C is shared and updated
  $U \leftarrow \emptyset$ ▷ Branches left unresolved
  for all $Q \in T$ **do**
    if $X = \emptyset$ **then**
      $U \leftarrow U \cup \{Q\}$
      continue
    end if
    $x \leftarrow$ ChooseVariable(Q, X)
    $(S_0, U_0, C) \leftarrow$ SolveByZeroing($Q \cup \{x\}, X \setminus \{x\}, \tau, C$)
    $(S_1, U_1, C) \leftarrow$ SolveByZeroing($Q \cup \{1 - xz\}, X \setminus \{x\}, \tau, C$) ▷ z is fresh
    $S \leftarrow S \cup S_0 \cup S_1$
    $U \leftarrow U \cup U_0 \cup U_1$
  end for
  return (S, U, C)
end function

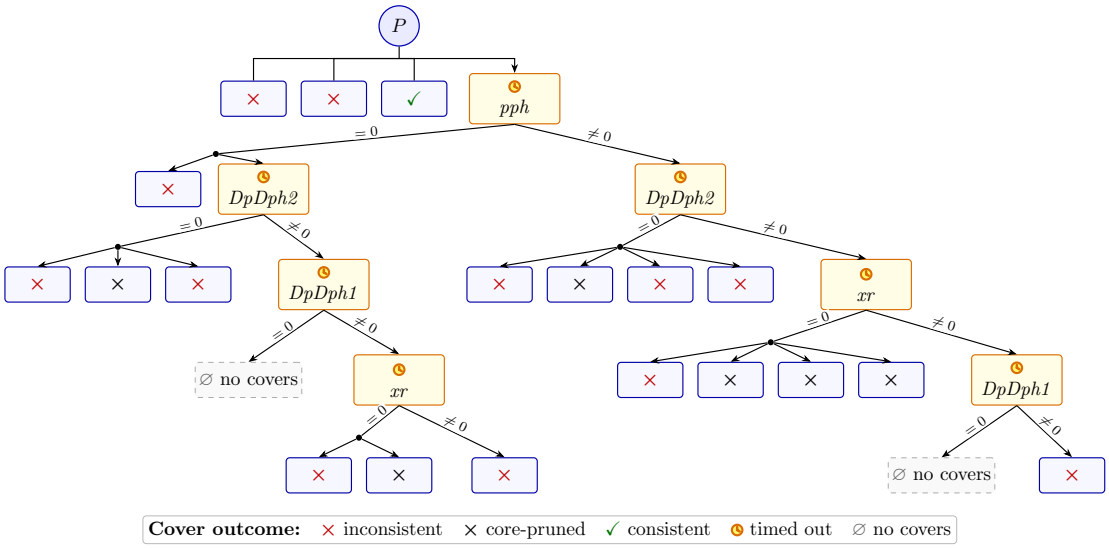


Figure 2: Recursion tree for `systems/system.txt`. Each cover is shown separately; ∅ marks a branch that produces no covers.

For each candidate variable x that is still available on the current branch, define

$$\begin{aligned} g_Q(x) &= \#\{q \in Q : x \text{ occurs in } q\}, \\ t_Q(x) &= \#\{(q,m) : q \in Q,\ m \text{ is a term of } q \deg_x(m) > 0\}, \\ e_Q(x) &= \sum_{q \in Q} \sum_{m \text{ a term of } q \deg_x(m) > 0} \deg_x(m). \end{aligned}$$

Its score is the triple

$$h_Q(x) = \big(g_Q(x), t_Q(x), e_Q(x)\big),$$

and scores are compared lexicographically.

4.2 Theoretical analysis of variable zeroing

Let Y be the set of variables in the input system, and let π_Y denote projection onto those coordinates. For any system Q encountered during the recursion, write

$$V_Y(Q) = \pi_Y(V(Q)).$$

This projection removes the inverse variables introduced along the path to Q.

We use the following contract for BranchingWithTimeout. If a call on Q returns a set of solutions S_Q and a set of unresolved branches T_Q, then

$$V_Y(Q) = S_Q \cup \bigcup_{R \in T_Q} V_Y(R). \tag{1}$$

Here and below, solver output is identified with the subset of $\overline{K}^Y$ that it represents. Equation (1) is the usual branching invariant: a timeout moves an unprocessed branch to T_Q instead of discarding it. We also assume, as in Theorem 3, that every learned unsatisfiable core is certified.

Lemma 4 (Correctness of a zeroing split) *Let $x \in Y$, and let z be a variable that does not occur in Q. Then*

$$V_Y(Q) = V_Y(Q \cup \{x\}) \cup V_Y(Q \cup \{1 - xz\}).$$

Proof 7 *Take $a \in V_Y(Q)$ and choose a point $\widetilde{a} \in V(Q)$ with $\pi_Y(\widetilde{a}) = a$. If $\widetilde{a}_x = 0$, then $a \in V_Y(Q \cup \{x\})$. If $\widetilde{a}_x \neq 0$, extend $\widetilde{a}$ by setting $z = \widetilde{a}_x^{-1}$. The resulting point satisfies $1 - xz = 0$, so $a \in V_Y(Q \cup \{1 - xz\})$. This proves the forward inclusion. The reverse inclusion follows because each child system contains every polynomial in Q.*

Theorem 4 (Variable zeroing) *Suppose Algorithm 2 returns (S, U, C) on input P. Then*

$$V(P) = S \cup \bigcup_{R \in U} V_Y(R).$$

In particular, if $U = \emptyset$, then $S = V(P)$.

Proof 8 *We prove that by showing that every call* SolveByZeroing(Q, X, τ, C) *returning (S', U', C') satisfies*

$$V_Y(Q) = S' \cup \bigcup_{R \in U'} V_Y(R). \tag{2}$$

The proof is by induction on $|X|$.

If $X = \emptyset$, the procedure cannot split an unresolved branch. It returns the solutions found by BranchingWithTimeout *and places every branch in T_Q into U'. Thus, Equation* (2) *is exactly Equation* (1)*.*

Now suppose $X \neq \emptyset$ and that the claim holds whenever fewer variables are available. Let (S_Q, T_Q) be the result of BranchingWithTimeout *on Q. For each $R \in T_Q$, the procedure chooses some $x \in X$ and recursively processes*

$$R_0 = R \cup \{x\} \qquad \text{and} \qquad R_1 = R \cup \{1 - xz\},$$

where z is fresh. Both calls use the smaller set $X \setminus \{x\}$. Lemma 4 gives

$$V_Y(R) = V_Y(R_0) \cup V_Y(R_1).$$

Substituting these decompositions into Equation (1) *shows that its right-hand side is the union of S' and the varieties indexed by U'. This is Equation* (2)*.*

Apply the claim to the initial call with $Q = P$. Since P contains no inverse variables, $V_Y(P) = V(P)$, which proves the stated identity. If U is empty, the union over U is empty and hence $S = V(P)$.

Now to show the run always terminates in finite time.

Theorem 5 *Zeroing-Branching 2 terminates in finite time.*

Proof 9 *The depth of the recursion tree is at most $|X|$ since for each call an unused variable is required and removed. The branching factor is bounded by the number of hitting sets of factors which is finite. The tree size is finite, and due to the timeout each hitting set is evaluated in finite time. Thus the algorithm terminates in finite time.*

We can infer the solutions of the hitting set the solver timed out on from its children's solutions. If all children are unsatisfiable, then so is the timeout evaluation. Otherwise its solution is the union of the children's solutions.

5 Experiments

5.1 Setup

We implemented Algorithm 1 in Rust 1.97.1 and used Symbolica 1.5 for polynomial arithmetic over the rationals. In all experiments, we solve the branches with a Gröbner-basis computation. The available implementations are M4GB, F5, GVW, Symbolica's built-in Gröbner-basis implementation, and Singular 4.4.1. A basis containing 1 certifies that the branch is unsatisfiable. We obtain an unsatisfiable core by iterative refinement as in Section 5 of [15]. The comparison also includes six direct whole-ideal Gröbner-basis configurations, three decomposition methods from Singular, Macaulay2 1.26.06, and a numerical method from HomotopyContinuation.jl under Julia 1.12.6.

We ran the benchmark on an Apple M4 Pro with 14 CPU cores and 24 GB of memory. Every table entry is a single run. Every method received 60 seconds per system.

For branching, T is end-to-end wall time and F is the time from the start of the factor-cover search to its first satisfiable cover. Factorization and input processing are therefore included in T but not in F. The complete measurements are stored in the appendix.

5.2 Baseline systems

The baseline has 20 generated systems in six families. Signed-distance cycles and affine-factor hypergraphs test many inexpensive covers. The overlapping random multiples use seeded dense quadratic factors that recur across equations, with no factor shared by every equation. In the gated systems, a single selector multiplies every equation of a Cyclic, Katsura, or dense quadratic system. This produces one easy selector cover and one nonlinear residual cover.

Family	Tested sizes	Variables/equations; covers
Signed-distance cycle	$n = 8, 10, 12$	$8/8, 10/10, 12/12$; $8, 8, 32$
Overlapping random multiples	$(n, p, k) = (4, 6, 2), (6, 8, 3), (8, 10, 3)$	n/n; $4, 16, 28$
Positive affine hypergraph	$(n, m, k) = (8, 6, 4), (10, 8, 5), (12, 10, 6)$	n/m; $63, 239, 833$
Gated Cyclic	orders $5, 6, 7$	$6/5, 7/6, 8/7$; 2 each
Gated Katsura and dense quadratics	Katsura $6, 7, 8$; dense $n = 5, 6, 7$	$8/7, 9/8, 10/9, 6/5, 7/6, 8/7$; 2 each
Square affine hypergraph	$(n, k) = (8, 4), (10, 5)$	n/n; $31, 304$

Table 1: The generated systems used for the baseline benchmark. The overlapping random multiples use seed 31415; both affine families use seed 42.

The overlapping-random-multiple runs disable interreduction so that the generated factor-incidence pattern reaches the cover enumerator unchanged. All other branching runs use interreduction. Table 2 selects the fastest completed branching backend and the fastest completed method in each baseline class. A branching entry gives T/F in seconds; the other entries give total wall time. "Timeout" means that every tested method in that class timed out.

System	Best branching T/F	Best direct GB	Best decomposition	Numerical
Signed distance, $n = 8$	Symb. 0.008/0.000	built-in 0.008	facstd 0.190	32.410
Signed distance, $n = 10$	Symb. 0.025/0.002	built-in 0.029	facstd 0.430	29.944
Signed distance, $n = 12$	Symb. 0.087/0.002	built-in 0.472	primdec 3.056	36.880
Overlapping $(4, 6, 2)$	F5 0.008/0.000	built-in 0.413	facstd 0.265	30.069
Overlapping $(6, 8, 3)$	F5 0.132/0.001	timeout	timeout	timeout
Overlapping $(8, 10, 3)$	Singular 2.931/0.010	timeout	timeout	timeout
Positive affine $(8, 6, 4)$	F5 0.008/0.000	timeout	facstd 0.368	50.934
Positive affine $(10, 8, 5)$	F5 0.020/0.000	timeout	facstd 1.836	timeout
Positive affine $(12, 10, 6)$	F5 0.067/0.000	timeout	facstd 53.318	timeout
Gated Cyclic-5	Symb. 0.009/0.000	built-in 0.008	facstd 0.181	35.054
Gated Cyclic-6	Singular 0.045/0.008	Singular 0.309	M2 2.620	37.314
Gated Cyclic-7	timeout	timeout	timeout	58.287
Gated Katsura-6	Singular 0.049/0.008	GVW 0.226	facstd 0.233	31.532
Gated Katsura-7	Singular 0.283/0.008	GVW 3.176	facstd 0.539	31.980
Gated Katsura-8	Singular 2.657/0.052	timeout	facstd 3.596	34.264
Gated dense, $n = 5$	Singular 0.029/0.008	M4GB 0.171	facstd 0.198	29.289
Gated dense, $n = 6$	Singular 0.546/0.008	M4GB 4.736	facstd 0.755	30.053
Gated dense, $n = 7$	Singular 42.020/0.017	timeout	facstd 48.820	32.261
Square affine $(8, 4)$	Singular 0.470/0.023	timeout	facstd 0.601	timeout
Square affine $(10, 5)$	Symb. 1.174/0.000	timeout	facstd 2.141	timeout

Table 2: Best completed result in each method class. Branching entries show wall time and first-solution latency; baseline entries show wall time. All times are in seconds.

The larger gated systems expose the cost of the nonlinear residual cover that is hidden by the easy selector cover. At Cyclic-7, every branching backend finds the selector component immediately but fails to exhaust both covers in 60 seconds; all direct Gröbner methods also time out, while the numerical method completes in 58.287 seconds. For Katsura-8, branching with Singular still completes in 2.657 seconds, compared with 3.596 seconds for facstd, and every direct method times out. Dense $n = 7$ is harder: branching with Singular takes 42.020 seconds and HomotopyContinuation.jl takes 32.261 seconds, while all direct methods time out. Despite this increase in total work, branching finds the first satisfiable selector cover within 0.052 seconds on every larger gated system.

5.3 Physical Systems

6 Conclusion

References

[1] Bruno Buchberger. Bruno Buchberger's PhD thesis 1965: An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal. *Journal of Symbolic Computation*, 41(3–4):475–511, 2006.

[2] David A. Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer, Cham, 4 edition, 2015.

[3] Jean-Charles Faugère. A new efficient algorithm for computing Gröbner bases (F4). *Journal of Pure and Applied Algebra*, 139(1–3):61–88, 1999.

[4] Jean-Charles Faugère. A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). In *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation*, pages 75–83. ACM, 2002.

[5] Patrizia Gianni, Barry Trager, and Gail Zacharias. Gröbner bases and primary decomposition of polynomial ideals. *Journal of Symbolic Computation*, 6(2–3):149–167, 1988.

[6] Hiroshi Kera, Yuki Ishihara, Yuta Kambe, Tristan Vaccon, and Kazuhiro Yokoyama. Learning to compute Gröbner bases. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[7] Martin Kreuzer and Lorenzo Robbiano. *Computational Commutative Algebra 1*. Springer, Berlin, 2000.

[8] João Marques-Silva, Inês Lynce, and Sharad Malik. Conflict-driven clause learning SAT solvers. In *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*, pages 131–153. IOS Press, 2009.

[9] João P. Marques-Silva and Karem A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[10] Ernst W. Mayr and Albert R. Meyer. The complexity of the word problems for commutative semigroups and polynomial ideals. *Advances in Mathematics*, 46(3):305–329, 1982.

[11] Dylan Peifer, Michael Stillman, and Daniel Halpern-Leistner. Learning selection strategies in Buchberger's algorithm. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7575–7585. PMLR, 2020.

[12] Andrew J. Sommese, Jan Verschelde, and Charles W. Wampler. Numerical decomposition of the solution sets of polynomial systems into irreducible components. *SIAM Journal on Numerical Analysis*, 38(6):2022–2046, 2001.

[13] Andrew J. Sommese and Charles W. Wampler, II. *The Numerical Solution of Systems of Polynomials Arising in Engineering and Science*. World Scientific, Singapore, 2005.

[14] Bernd Sturmfels. *Solving Systems of Polynomial Equations*, volume 97 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI, 2002.

[15] Xiaojing Sun, Ioana Ilioaea, Priyank Kalla, and Florent Enescu. Finding unsatisfiable cores of a set of polynomials using the Gröbner basis algorithm. In *Principles and Practice of Constraint Programming*, volume 9892 of *Lecture Notes in Computer Science*, pages 859–875. Springer, 2016.

A Appendix